Real-Time Stock Market Data visualization

Introduction

It’s a full-stack Python-based web app that helps users visualize real-time and historical stock market data.”

“We built it with the goal of helping users track the performance of multiple stocks, get real-time updates, receive alerts, and compare historical trends — all in a visual and interactive way.”

Problem Statement

Raw financial data is difficult to interpret without visuals.

Many beginner investors lack tools for accessible real-time insights.

A need exists for a simple dashboard with dynamic and static visualizations.

Project Goals

Scrape and serve live + historical stock data.

Build a clean API using Python and Flask.

Visualize data using interactive and static charts.

Make financial analysis simple and approachable.

Tech Stack

Backend: Python (Flask)

Web Scraping: Selenium, BeautifulSoup, Requests

Data Handling & Analysis:

Visualization: Chart.js

Background Tasks: schedule module

Frontend: HTML, CSS, JavaScript

scraper.py

Core Functionality

- Purpose: The scraper is responsible for gathering stock data (current price, historical data) by making requests to a stock data website (Investing.com).

Steps:

- The scraper uses the requests library to send a request to the website.
- If the website requires dynamic content, the scraper uses Selenium and BeautifulSoup to render the page and extract the stock data.
- The scraper retrieves data such as the current price, historical prices, open/high/low values, and volume.

stock_routes.py

Core Functionality

- Purpose: The routes file defines API endpoints that handle requests for stock data.

Steps:

- It uses the Blueprint class in Flask to define routes related to stocks.

The routes include:

- /api/stocks/ — Returns a list of all stocks.
- /api/stocks/<id>/price — Returns the current price for a specific stock.
- /api/stocks/<id> — Returns detailed data (historical prices, volume, etc.) for a specific stock

stock_controllers

Core Functionality

- Purpose: The controller file manages the business logic, including interacting with the scraper, caching the results, and handling the API responses.

Steps:

- The controller uses threads (or better, a ThreadPoolExecutor) to scrape stock data in the background, without blocking the API responses.
- It manages a cache to store stock prices and updates them at regular intervals (120 seconds).
- The controller uses the functions from the scraper to fetch stock data and present it to the API routes.

System Workflow

How It Works Together

- The Flask server handles incoming requests and routes them to the appropriate controller functions.
- The controller uses the scraper to fetch stock data and store it in a cache.
- The background threads ensure that the cache is updated regularly, keeping the stock data fresh.
- The routes provide endpoints for retrieving stock prices and historical data.

- GET /api/stocks/ to list all companies.
- GET /api/stocks/<id>/price to fetch the current price.
- GET /api/stocks/<id> to fetch detailed data for a stock.

Key Features

- Stock Price Caching: The API caches stock prices to avoid fetching the same data repeatedly. This improves performance and reduces load on the external website.
- Background Scraping: We use background threads to periodically fetch the latest stock prices and update the cache. This ensures that the data is always fresh, without blocking the main thread of the application.

The API provides:

- A list of companies with their basic information.
- The current stock price for a specific company.
- Detailed stock data for a specific company, including historical prices, open, high, low, volume, and percentage change.

Challenges Faced

Handling dynamic websites with Selenium

Ensuring scraper speed and stability

Managing thread-safe cache updates

Rendering seamless chart transitions in frontend

Future Enhancements

Technical Indicators:

- Add indicators like SMA, EMA, RSI, and MACD for better stock trend analysis.

Stock Buying Simulation:

- Enable users to virtually buy/sell stocks, manage a portfolio, and track performance.

Real-Time Updates:

- Use WebSockets or SSE for live price feeds.

Conclusion

- Built a scalable backend system to scrape, process, and serve real-time stock data using Flask and Selenium.
- Integrated Bright Data proxy API for reliable data fetching and caching to reduce load and response time.
- Designed API routes to fetch company list, current stock prices, and detailed historical data.
- Laid a strong foundation for future upgrades like technical indicators and virtual trading features.

Thank You